

Cel bonusu:

Nie uczę się listy haseł.
Uczę się, jaki problem pojawia się w modelu i jaką decyzją go rozwiązuję.

W lekcji 03 zbudowaliśmy prosty model:

```
fact_sales + dim_customer + dim_product -> Gold
```

To działa, dopóki model jest mały.

Bonus pokazuje, co dzieje się dalej:

1. Klient zmienia region.
2. Dochodzą zwroty i web events.
3. Pojawiają się różne typy metryk.
4. Produkt ma wiele tagów.
5. Każdy dashboard chce własną definicję revenue.

Każde nowe pojęcie jest odpowiedzią na jeden z tych problemów.

Punkt Startowy

Mamy prostą gwiazdę:

```
          dim_customer
          |
dim_product ---- fact_sales
```

fact_sales:

jeden rekord = jedna pozycja opłaconego zamówienia

dim_customer:

jeden rekord = jeden klient

Na tym poziomie zwykle **customer id** i **product id** wyglądają wystarczająco.

Problem 1: Klient Zmienia Region

Założmy:

```
customer_id = 42  
region do 2026-01-04: north  
region od 2026-01-04: central
```

Pytanie modelarskie:

```
Czy sprzedaż z 2026-01-01 ma zostać w north?  
Czy ma się przepisać na aktualny central?
```

Jeśli historia nie ma znaczenia, wystarczy nadpisać region.

Jeśli historia ma znaczenie, potrzebujesz wersji klienta w czasie.

Rozwiązanie: SCD

SCD = Slowly Changing Dimension.

To nazwa na strategię obsługi zmian w dimension table.

Nie chodzi o to, że wymiar zmienia się wolno technicznie.

Chodzi o pytanie:

Co robimy, gdy opis klienta albo produktu zmienia się w czasie?

Najważniejsze Typy SCD

Na start wystarczy znać mapę:

```
Type 0 = nie zmieniaj wartości  
Type 1 = nadpisz starą wartość  
Type 2 = dodaj nowy wiersz i zachowaj historię  
Type 3 = trzymaj poprzednią wartość w dodatkowej kolumnie
```

W praktyce najczęściej spotkasz Type 1 i Type 2.

SCD Type 1

Type 1 nadpisuje starą wartość.

customer_id	region
42	central

Zaleta:

prosto, jeden rekord na klienta

Koszt:

historia regionu north znika

Użyj tego, gdy historia nie ma znaczenia albo poprawiasz błąd.

SCD Type 2

Type 2 dodaje nową wersję rekordu zamiast nadpisywać starą.

customer_sk	customer_id	region	valid_from	valid_to
1001	42	north	2025-01-01	2026-01-04
1002	42	central	2026-01-04	null

To pozwala zapytać:

Jaki region miał klient w dniu zamówienia?

Rozwiązanie: Business Key Vs Surrogate Key

Business key pochodzi z biznesu albo source systemu.

```
customer_id = 42
```

Ale przy historii jeden `customer_id` może mieć kilka wersji.

Surrogate key to techniczny klucz wersji w modelu:

customer_sk	customer_id	region
1001	42	north
1002	42	central

`customer_id` mówi:

To ten sam klient biznesowo.

`customer_sk` mówi:

To konkretna wersja klienta w czasie.

Surrogate Key Przy SCD Type 2

SCD Type 2 zachowuje historię wymiaru.

customer_sk	customer_id	region	valid_from	valid_to
1001	42	north	2025-01-01	2026-01-04
1002	42	central	2026-01-04	null

Fact table może wtedy wskazać właściwą wersję klienta.

Najważniejsza intuicja:

```
business key identyfikuje obiekt
surrogate key identyfikuje wersję obiektu
```

Nie każdy model potrzebuje surrogate keys od razu.

Najczęściej pojawiają się, gdy potrzebujesz historii albo integrujesz wiele źródeł.

Problem 2: Dochodzą Inne Fact Tables

Najpierw mieliśmy tylko sprzedaż:

```
fact_sales
```

Potem dochodzą:

```
fact_returns  
fact_web_events  
fact_support_tickets
```

Każdy fact może mieć datę, klienta i produkt.

Ryzyko:

```
każdy zespół tworzy własne dim_date albo dim_customer
```

Wtedy raporty nie mówią tym samym językiem.

Rozwiązanie: Conformed Dimension

Conformed dimension to wspólny wymiar używany przez wiele fact tables.

Przykład:

```
dim_date
  używa fact_sales
  używa fact_returns
  używa fact_web_events
```

Albo:

```
dim_customer
  używa sprzedaż
  używa support
  używa marketing
```

Po co?

```
żeby różne raporty miały tę samą definicję daty, klienta, regionu
albo segmentu
```

Conformed dimension to decyzja o spójności między modelami.

Problem 3: Nie Każdy Fact Opisuje To Samo

`fact_sales` opisuje zdarzenie sprzedaży.

Ale nie każdy fact table jest zdarzeniem.

Przykłady:

```
sprzedaż produktu  
stan magazynu na koniec dnia  
proces zamówienia od złożenia do dostawy
```

To są różne typy faktów, bo mają inny grain i inny sens biznesowy.

Rozwiązanie: Typy Fact Tables

Najczęstsze typy fact tables:

Transaction fact

jeden rekord = jedno zdarzenie

przykład: pozycja zamówienia

Periodic snapshot fact

jeden rekord = stan w okresie

przykład: inventory per dzień

Accumulating snapshot fact

jeden rekord = proces z etapami

przykład: order lifecycle

W lekcji 03 robimy transaction fact:

```
fact_sales: jeden rekord = jedna pozycja opłaconego zamówienia
```

Problem 4: Nie Każdą Metrykę Sumujesz Tak Samo

Revenue jest proste:

```
SUM(revenue)
```

Ale inne metryki bywają zdradliwe:

```
inventory  
conversion_rate  
margin_percent  
average_price
```

Pytanie modelarskie:

```
Czy tę metrykę wolno sumować po dowolnym wymiarze?
```

Jeśli nie, musisz opisać regułę liczenia.

Rozwiązanie: Additive, Semi-Additive, Non-Additive

Te trzy słowa mówią tylko jedno:

Czy mogę bezpiecznie zrobić `SUM(metric)`?

Additive

możesz sumować prawie wszędzie

przykład: revenue

`SUM(revenue)` po dniach, produktach i regionach ma sens

Semi-additive

możesz sumować po części wymiarów, ale nie po czasie

przykład: inventory na koniec dnia

`SUM(inventory)` po magazynach ma sens,

ale `SUM(inventory)` po 30 dniach zwykle zawyża wynik

Non-additive

nie sumuj samej metryki

przykład: `conversion_rate`, `margin_percent`

licz z części składowych: `SUM(numerator) / SUM(denominator)`

Błąd interview:

`AVG(procentów)` bez sprawdzenia wag

Lepszy wzorec:

Problem 5: W Fact Są Identyfikatory Bez Wymiaru

W `fact_sales` często trzymamy:

```
order_id
```

`order_id` jest ważny do debugowania i drill-down.

Ale osobna `dim_order` nie zawsze ma sens.

Jeśli `order_id` nie ma własnych atrybutów opisowych, może zostać w fact table.

Rozwiązanie: Degenerate Dimension

Degenerate dimension to identyfikator biznesowy trzymany w fact table bez osobnej dimension.

Przykład:

```
fact_sales.order_id
```

To nadal pomaga filtrować albo debugować.

Ale nie tworzymy osobnej tabeli tylko po to, żeby mieć jedną kolumnę `order_id`.

W lekcji 03 `order_id` w `fact_sales` jest właśnie takim praktycznym identyfikatorem.

Problem 6: Produkt Ma Wiele Tagów

Założmy:

```
Keyboard ma tagi: accessories, office, bestseller  
Mouse ma tagi: accessories, gaming
```

Relacja nie jest prosta:

```
jeden produkt -> wiele tagów  
jeden tag -> wiele produktów
```

To jest many-to-many.

Zwykły join może łatwo podwoić metryki, jeśli nie wiesz, na jakim grainie liczysz.

Rozwiązanie: Bridge Table

Bridge table pomaga przy relacji many-to-many.

Nie wkładasz listy tagów do `dim_product` jako jeden tekst.

Robisz osobną tabelę połączeń:

```
dim_product
product_id  product_name
10          Keyboard

dim_tag
tag_id      tag_name
1           accessories
2           office

bridge_product_tag
product_id  tag_id
10          1
10          2
```

Bridge mówi:

```
produkt 10 ma tag 1
produkt 10 ma tag 2
```

Jak Bridge łączy Się Z Fact

Ścieżka joinu wygląda tak:

```
fact_sales
-> dim_product
-> bridge_product_tag
-> dim_tag
```

Przykład pytania:

```
Pokaż sprzedaż produktów z tagiem accessories.
```

Wtedy filtrujesz po `dim_tag`, ale revenue nadal pochodzi z `fact_sales`.

Uwaga:

```
Jeśli produkt ma 3 tagi i zsumujesz revenue po tagach,  
ten sam sale może pojawić się 3 razy.
```

Bridge table jest potrzebna, gdy relacja nie jest 1 do wielu albo gdy zwykły join mógłby podwoić metryki.

W miesiącu 01 tylko nazwij ten problem. Nie implementuj bridge tables, jeśli case ich nie wymaga.

Problem 7: Każdy Dashboard Liczy Revenue Inaczej

Jedna osoba liczy:

```
SUM(quantity * unit_price)
```

Druga zapomina o filtrze:

```
status = 'paid'
```

Trzecia używa gotowego Golda, ale nie wie, co w nim jest.

Problem:

```
metryka nie ma jednego właściciela i jednej definicji
```

Rozwiązanie: Semantic Layer

Semantic layer to warstwa definicji metryk i pojęć biznesowych.

Gold table daje dane:

```
gold_daily_sales.total_revenue
```

Semantic layer mówi, co to znaczy:

```
paid_revenue = SUM(quantity * unit_price)
WHERE status = 'paid'
```

Po co?

żeby dashboardy, analitycy i aplikacje używały tej samej definicji metryki

Gold Vs Semantic Layer

Gold table:

gotowa tabela pod konkretny use case

Semantic layer:

centralna definicja metryk i wymiarów

Gold może być wejściem do semantic layer.

Semantic layer może też budować zapytania na star schema.

Nie zastępują się idealnie:

Gold: co czyta dashboard?

Semantic layer: co dokładnie znaczy metryka?

Przykłady Semantic Layer

To może być osobne narzędzie albo warstwa w istniejącym stacku.

Przykłady narzędzi albo idei:

```
dbt metrics / MetricFlow  
LookML  
Cube  
warstwa metryk w BI
```

Nie musisz tego implementować w miesiącu 01.

Masz rozumieć problem:

```
oficjalna definicja metryki powinna mieszkać w jednym  
kontrolowanym miejscu
```

Co Jest Dla Nas Teraz Ważne

W miesiącu 01 obowiązkowe są:

```
grain
fact vs dimension
star schema
Gold
checks
reliability basics
```

Bonusowo warto znać:

```
surrogate key
conformed dimension
fact table types
additive/semi-additive/non-additive
semantic layer
bridge table
```

Nie musisz jeszcze tego wszystkiego implementować.

Stały Wzorzec Myślenia

Nie pytaj najpierw:

Czy dodać surrogate key?
Czy zrobić semantic layer?

Pytaj:

Jaki problem mam w modelu?

Potem dobierz pojęcie:

historia wersji obiektu -> surrogate key + SCD Type 2
wspólny język wielu factów -> conformed dimension
proces zamiast zdarzenia -> inny typ fact table
metryka procentowa -> sprawdź additive/non-additive
many-to-many -> bridge table
wiele definicji metryki -> semantic layer

Mini-Checklist

Przy nowym modelu zapytaj:

- ☐ Jaki jest grain fact table?
- ☐ Czy mam business key i czy potrzebuję surrogate key?
- ☐ Czy dimension powinna być conformed?
- ☐ Jaki typ fact table buduję?
- ☐ Czy metryka jest additive?
- ☐ Czy mam many-to-many i potrzebuję bridge table?
- ☐ Czy definicja metryki powinna trafić do semantic layer?

Jeśli nie umiesz odpowiedzieć, model nie jest jeszcze gotowy do skali.